

ELTE EÖTVÖS LORÁND UNIVERSITY

INPUT MANAGEMENT IN BLAZOR

Modern web application development in .NET

Lecture 9

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

DATA BINDING

Blazor supports *two-way data binding*, allowing components to interact with user input and keep the UI and the underlying C# property in sync.

- e.g.:

```
<label for="fullNameInput">Full Name:</label>
<input id="fullNameInput" @bind="fullName" />
<label for="birthYearInput">Birth Year:</label>
<input id="birthYearInput" @bind="birthYear" />

@code {
    string fullName = "Loránd Eötvös";
    int birthYear = 1848;
}
```

FORMS IN BLAZOR

Input fields can be grouped into forms, for which the **`@onsubmit`** event can be binded and handled.

- e.g. (*HTML segment of component*):

```
<form method="post" onsubmit="@HandleSubmit">
  <label for="fullNameInput">Full Name:</label>
  <input id="fullNameInput" @bind="fullName" />
  <label for="birthYearInput">Birth Year:</label>
  <input id="birthYearInput" @bind="birthYear" />
  <button type="submit">Submit</button>
</form>
```

FORMS IN BLAZOR

- e.g. (*C# code segment of component*):

```
@code {
    string fullName = "Loránd Eötvös";
    int birthYear = 1848;
    private void HandleSubmit() {
        // … do form processing …
    }
}
```

- when required, the event handler method can be asynchronous
  - should return a Task in that case

THE EDITFORM COMPONENT

Blazor provides the **`EditForm`** component for managing entire forms, including validation and submission

- **`EditForm`** seamlessly integrates with data models, validation logic, and UI rendering
- It binds an object – a (view)model – to a form, while manages submission and validation
  - The **`Model`** parameter represents the data-bound object
- Handles submit events with **`OnValidSubmit`**, **`OnInvalidSubmit`**, or **`OnSubmit`**. (The latter is executed regardless of the validation status.)
- Supports both client-side and server-side models

INPUT COMPONENTS

Blazor provides several built-in input components for form fields

- These components automatically bind to model properties and support validation
  - **`InputText`**, **`InputNumber`**, **`InputSelect`**, **`InputCheckbox`**, etc.
  - File management is also supported with **`InputFile`**
- Each input uses the **`@bind-Value`** directive to link to a model property
- E.g. (custom complex type):

```
public class Person
{
    public string FullName { get; set; }
    public int BirthYear { get; set; }
}
```

FORM SUBMISSION

- E.g. (Razor component):

```
<EditForm Model="@person" OnValidSubmit="HandleSubmit">
    <InputText @bind-Value="person.FullName" />
    <InputNumber @bind-Value="person.BirthYear" />
    <button type="submit">Submit</button>
</EditForm>

@code {
    private Person person = new Person();
    private void HandleSubmit() {
        // process the form …
    }
}
```

FORM VALIDATION

Blazor supports form validation using standard *.NET Data Annotations*, which we already used for viewmodel validation in MVC applications.

- The properties in the (view)model can be decorated with attributes like **`[Required]`**, **`[StringLength]`**, **`[Range]`**, **`[Compare]`**, **`[Url]`**, **`[EmailAddress]`**, [**`Phone`**], **`[RegularExpression]`**, etc.
- Blazor automatically validates the form before submission, when **`DataAnnotationsValidator`** is added to the form
- Validation messages can be displayed using **`ValidationSummary`** (displays all validation messages) or **`ValidationMessage`** (shows validation for a specific field)

FORM VALIDATION

- E.g. (custom complex type):

```
public class Person
{
    [Required]
    public string Name { get; set; }

    [Range(1000, 3000)]
    public int BirthYear { get; set; }
}
```

FORM VALIDATION

- E.g. (Razor component):

```
<EditForm Model="@person" OnValidSubmit="HandleSubmit">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <InputText @bind-Value="person.FullName" />
    <ValidationMessage For="@(() => person.FullName)" />

    <InputNumber @bind-Value="person.BirthYear" />
    <ValidationMessage For="@(() => person.BirthYear)" />

    <button type="submit">Submit</button>
</EditForm>
```

CUSTOM FORM VALIDATION

Further *custom validator* attribute classes can be inherited from **`ValidationAttribute`**, implementing the **`IsValid()`** method.

Alternatively, Blazor also allows custom validation logic beyond data annotations.

- The **`IValidatableObject`** interface can be implemented for more advanced rules.
- This allows to write logic that depends on multiple fields or implements complex rules.

CUSTOM FORM VALIDATION

- e.g.:

```
public class Person : IValidatableObject
{
  public string FullName { get; set; }
  public int BirthYear { get; set; }

  public IEnumerable<ValidationResult> Validate(
    ValidationContext context)
  {
    if (!Name.Contains(" ")) {
      yield return new ValidationResult(
        "One-part names now allowed.", new[] { nameof(Name) });
    }
  }
}
```

JAVASCRIPT INTEROP IN BLAZOR

Blazor allows C# code to run in the browser via WebAssembly

- However, some functionality is only available through JavaScript (JS)
- To bridge this gap, Blazor provides JavaScript interoperability (JS Interop)
    - Enables calling JavaScript functions from C#
    - Also allows calling C# methods from JavaScript
    - Essential for integrating existing JS libraries or browser APIs

WHEN TO USE JAVASCRIPT INTEROP

While Blazor covers many common scenarios, some require JS. Examples include:

- Accessing browser-specific APIs
  - e.g., cookies, local storage, geolocation
  - NuGet packages could provide a better alternative (e.g. *Blazored.LocalStorage*, *Blazor.GeoLocation*), hiding the JS Interop
- Using third-party JavaScript libraries
- Manipulating the DOM directly (which is not idiomatic in Blazor)

Use JS Interop only when necessary, to maintain maintainability!

CALLING JAVASCRIPT FROM C#

Blazor provides **`IJSRuntime`** service to invoke JavaScript functions

- Use **`InvokeAsync<T>("functionName", args...)`**
  - Alternatively, **`InvokeVoidAsync()`** should be used when no return value is expected
- The function must be globally accessible in JS (e.g., in window)
- Supports async calls and returns a **`ValueTask<T>`**
- E.g.:

```
@inject IJSRuntime JS
...
await JS.InvokeAsync<string>("alert", "Hello from C#");
```

CALLING C# FROM JAVASCRIPT

C# methods can also be invoked from JavaScript

- Requires marking the C# method with [JSInvokable]
- The method must be static, or the instance must be passed explicitly
- Useful for event callbacks, timer completions, etc.
- E.g. (in C# code):

```
[JSInvokable]
public static void ShowMessage(string message) { … }
```

- E.g. (in JS code):

```
DotNet.invokeMethodAsync('AssemblyName’,
  'ShowMessage', 'Hello Blazor');
```

JAVASCRIPT INTEROP IN BLAZOR

Best practices and considerations:

- Minimize interop use to keep Blazor code clean and maintainable
- Always validate and sanitize inputs to/from JavaScript
- Prefer **`InvokeVoidAsync`** for functions that don't return values
- Keep JS files organized and clearly separate interop functions
- Use **`try-catch`** around interop calls to handle JS errors gracefully